REVISION CHEAT SHEET:

SQL Server Query Performance and Execution Plans

COMPLETE INTERVIEW-READY NOTES - PLANS - OPTIMIZER - CARDINALITY - OPERATORS - STATISTICS - SARGABILITY - QUERY STORE

TOPIC & SUBTOPIC INDEX

Compact study order: Topic Structure -> Query Performance -> Execution Plan -> Estimated Execution Plan -> Actual Execution Plan -> Estimated vs Actual Plan -> Query Optimizer. Related details continue in the same flow without unnecessary blank pages.

Topic Structure	Query Performance	Execution Plan
Estimated Execution Plan	Actual Execution Plan	Estimated vs Actual Plan
Query Optimizer	Cardinality Estimation	Index Seek
Index Scan	Seek vs Scan	Key Lookup
Sort Operator	Join Algorithms	Nested Loops
Hash Match	Merge Join	Statistics
SARGability	Parameter Sniffing	Parameter Sensitivity
PSP Optimization	Query Store	SET STATISTICS IO
SET STATISTICS TIME	Practical Query Tuning	Execution Plan Properties
Performance Relationships	Common Beginner Mistakes	Beginner-to-Advanced Analysis
Complete Performance Flow		

Topic Structure

EXAMPLE / FLOW

```
Query Performance
    →
Execution Plans
    →
Estimated Execution Plan
Actual Execution Plan
    →
Query Optimizer
    →
Cardinality Estimation
    →
Execution Plan Operators
        →
        Index Seek
        Index Scan
        Key Lookup
        Sort
        Nested Loops
        Hash Match
        Merge Join
    →
Statistics
    →
SARGability
    →
Parameter Sniffing / Parameter Sensitivity
    →
Query Store
    →
```

```
SET STATISTICS IO
SET STATISTICS TIME
    →
Practical Query Tuning
```

The examples below use your **FundraisingDb**, especially **fundraising.Gift**. That table contains **GiftId**, **TenantId**, **ConstituentId**, **CampaignId**, **GiftDate**, **TotalAmount**, and other useful columns for studying query performance.

Query Performance

Definition

Query performance means how efficiently SQL Server executes a SQL statement.

A query may return the correct result but still be poorly performing because it reads too many pages, consumes too much CPU, waits too long, uses too much memory, performs unnecessary sorting, or chooses an inefficient execution plan.

Example:

SQL

```sql
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

The SQL is simple, but its performance depends on things such as:

EXAMPLE / FLOW

```
Indexes
Statistics
Number of rows
Data distribution
Execution plan
Memory
CPU
I/O
Parameter values
```

Important Performance Measurements

When tuning a query, do not look only at execution time.

Important measurements include:

EXAMPLE / FLOW

```
Elapsed Time
CPU Time
Logical Reads
Physical Reads
Rows Processed
Estimated Rows
Actual Rows
Memory Grant
Sort / Hash Spills
Executions of Operators
Blocking / Waiting
```

A query taking one second while reading millions of pages may still have a serious design problem.

A query taking two seconds but reading only a small amount of data might simply be waiting on another process.

Query Performance Flow

EXAMPLE / FLOW

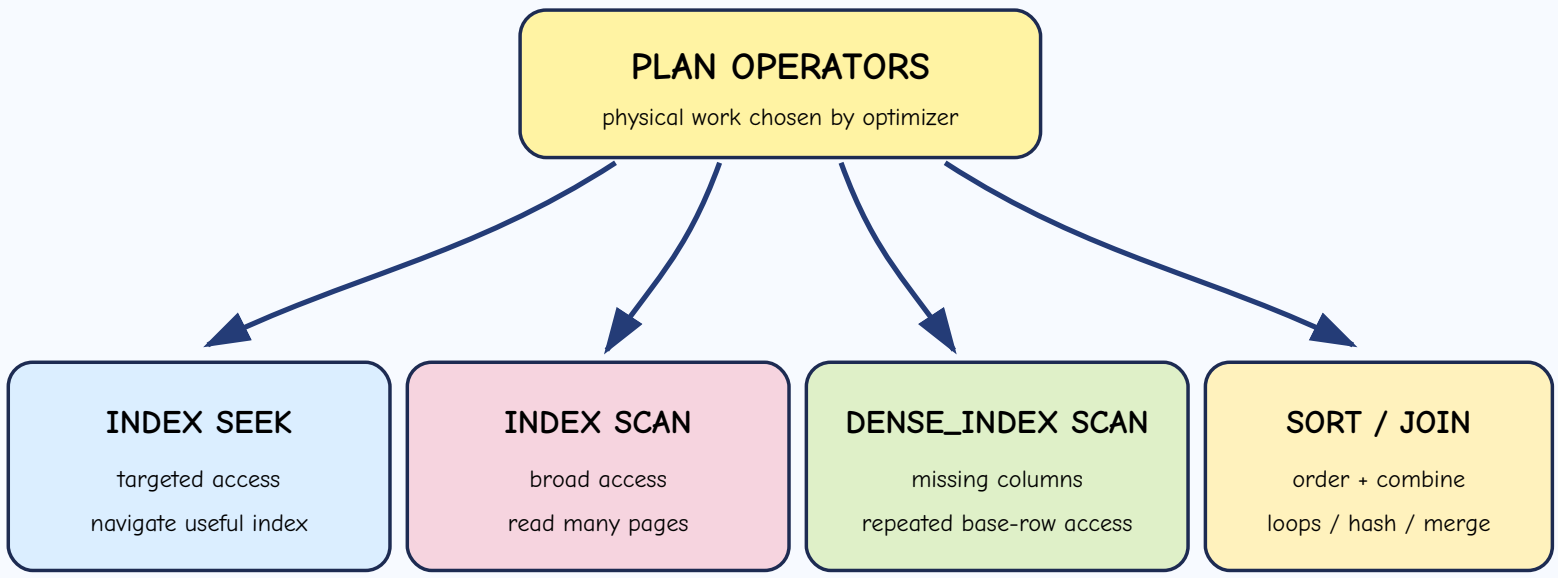
```
SQL Query
    →
Parsing
    →
Optimization
    →
Execution Plan
    →
Read / Join / Sort / Aggregate Data
    →
Return Result
```

SQL Server's Query Optimizer analyzes the query, schema, indexes, and statistics and builds a plan for executing the statement.

Execution Plan

3D DIAGRAM

Execution Plan Operator Family



Plan operators perform access, lookup, ordering, and join work chosen for the query.

Definition

An **execution plan** describes how SQL Server intends to execute or actually executed a SQL statement.

It can show:

EXAMPLE / FLOW

```
Which table is accessed first
Which indexes are used
Whether rows are scanned or sought
How tables are joined
Whether sorting happens
Estimated number of rows
Actual number of rows
Operator costs
Warnings
Memory information
```

An execution plan is one of the most important tools for SQL Server performance tuning.

Simple Example

Query:

```sql
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

Possible plan:

```
Index Seek
    →
SELECT
```

Another possible plan:

```
Clustered Index Scan
    →
Filter
    →
SELECT
```

Both queries return the same data.

The difference is **how SQL Server finds that data**.

Execution Plan Flow

For the diagrams in these notes, I will show the logical flow left-to-right:

```
Read Gift
    →
Filter Constituent
    →
Return Rows
```

In the graphical plan displayed by SQL Server Management Studio, data commonly flows from the operators on the right toward operators on the left.

So when looking at an SSMS graphical execution plan, you normally begin near the **right side** and follow the data toward the final SELECT.

Estimated Execution Plan

Definition

An **Estimated Execution Plan** shows the execution plan SQL Server expects to use **without actually executing the query**.

The plan is generated using information available during compilation, such as:

```
Indexes
Statistics
Query predicates
Join conditions
Cardinality estimates
Database settings
```

Because the query does not execute, an estimated plan does not contain actual runtime row counts or actual runtime resource information.

Example

In SSMS:

EXAMPLE / FLOW

```
Query
    →
Display Estimated Execution Plan
    →
Optimizer compiles query
    →
Plan displayed
```

The SQL statement itself is not executed.

When Estimated Plans Are Useful

Estimated plans are useful when:

- you do not want to execute an expensive query;
- the query changes data;
- you want to inspect a possible plan before running it;
- production execution would be dangerous;
- you want to examine index and join choices.

Example:

SQL

```
DELETE
FROM fundraising.Gift
WHERE GiftDate < '2020-01-01';
```

You might want to inspect the estimated plan without actually deleting anything.

Important Limitation

Suppose the optimizer predicts:

EXAMPLE / FLOW

```
Estimated Rows = 50
```

but the real query would return:

EXAMPLE / FLOW

```
Actual Rows = 500,000
```

The estimated plan alone cannot show this runtime difference.

That is why the **Actual Execution Plan** is usually more useful when diagnosing a query that you are able to safely execute.

Actual Execution Plan

Definition

An **Actual Execution Plan** contains the compiled execution plan together with runtime information collected while the query actually executes.

It may show:

EXAMPLE / FLOW

```
Estimated Rows
Actual Rows
Actual Executions
Elapsed Time
CPU information
Runtime warnings
Memory information
Spills
```

The actual plan becomes available after query execution finishes.

Example Flow

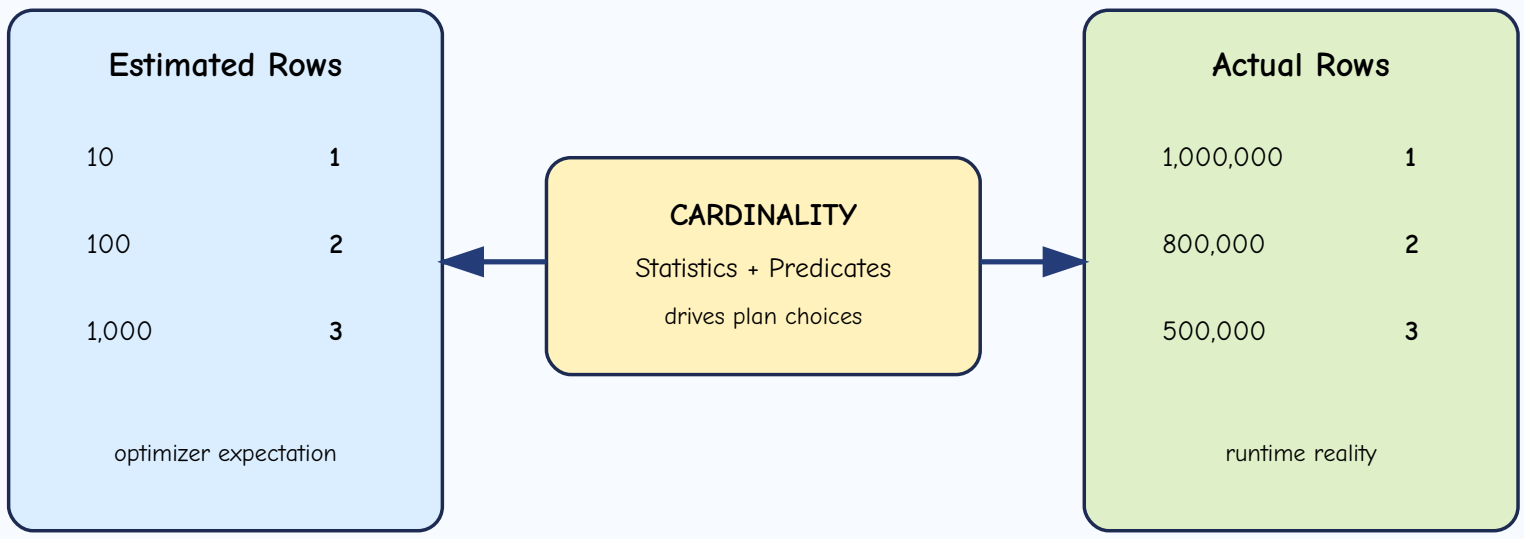
EXAMPLE / FLOW

```
Query
    →
Optimizer creates plan
    →
Query executes
    →
Runtime information collected
    →
Actual Execution Plan displayed
```

Why Actual Rows Are Important

3D DIAGRAM

Estimated Rows vs Actual Rows



CARDINALITY divides rows into independent groups; ORDER BY controls the ranking inside each group.

Suppose an operator shows:

EXAMPLE / FLOW

```
Estimated Rows = 10
Actual Rows    = 1,000,000
```

That is a major estimation difference.

SQL Server may have selected:

EXAMPLE / FLOW

```
Nested Loops
```

because it expected only 10 rows.

If SQL Server had known one million rows would appear, a different join such as a Hash Match might have been more appropriate.

Large estimated-versus-actual row differences are therefore an important tuning clue.

Estimated Plan vs Actual Plan

Estimated Plan	Actual Plan
Query is not executed	Query actually executes
Shows estimated rows	Shows estimated and actual rows

Estimated Plan	Actual Plan
No real runtime metrics	Includes runtime information
Safer for dangerous queries	Better for performance diagnosis
Generated during compilation	Contains compile plan + execution information

Do not think of them as two completely different plans.

SQL Server produces a compiled plan; the actual execution plan adds execution context and runtime information to it.

Query Optimizer

Definition

The **Query Optimizer** is the SQL Server component responsible for choosing an efficient way to execute a query.

You write:

SQL

```
SELECT
    c.DisplayName,
    g.TotalAmount
FROM constituent.Constituent AS c
INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId = c.ConstituentId
WHERE g.TotalAmount >= 1000;
```

You do not normally tell SQL Server:

EXAMPLE / FLOW

```
Read Gift first
Use this exact index
Use Nested Loops
Then access Constituent
```

Instead, you describe the required result.

The Query Optimizer decides how to retrieve it.

Optimizer Inputs

Conceptually:

EXAMPLE / FLOW

```
SQL Query
+
Indexes
+
Statistics
+
Table Information
+
Constraints
+
Parameter Values
+
Database Settings
    →
Query Optimizer
    →
Execution Plan
```

SQL Server uses a **cost-based optimizer**, meaning candidate plan choices are evaluated using estimated processing costs. Cardinality-the estimated number of rows flowing through operators-is one of the major inputs into those cost calculations.

Optimizer Decisions

The optimizer may decide:

EXAMPLE / FLOW

```
Index Seek vs Scan

Nested Loops vs Hash Match vs Merge Join

Which table to access first

Whether sorting is required

Whether parallel execution is worthwhile

How much memory to request

Which indexes are useful
```

Important Rule

The optimizer is not trying to find a mathematically perfect plan after examining every possible plan.

For complex queries there can be enormous numbers of possibilities.

SQL Server uses optimization strategies and heuristics to find a sufficiently good plan while keeping compilation time reasonable.

Cardinality Estimation

Definition

Cardinality Estimation, usually called **CE**, means predicting how many rows an operator will process or return.

Example:

SQL

```sql
SELECT *
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

Before running the query, SQL Server needs to estimate:

EXAMPLE / FLOW

```
How many Gift rows belong to ConstituentId 1001?
```

Possible estimate:

EXAMPLE / FLOW

```
Estimated Rows = 25
```

The real result may later be:

EXAMPLE / FLOW

```
Actual Rows = 27
```

That is a good estimate.

Why Cardinality Matters

Cardinality affects many optimizer decisions.

EXAMPLE / FLOW

```
Estimated Rows
    →
Join Type
    →
Index Access
    →
Memory Grant
```

```
    →
Sort Strategy
    →
Parallelism
    →
Execution Cost
```

More accurate row estimates usually give the optimizer a better chance of choosing an efficient plan.

Good Estimate

EXAMPLE / FLOW

```
Estimated Rows = 1,000
Actual Rows    = 1,050
```

The optimizer had a reasonably accurate understanding of the data.

Poor Estimate

EXAMPLE / FLOW

```
Estimated Rows = 10
Actual Rows    = 500,000
```

Possible consequences:

EXAMPLE / FLOW

```
Wrong join algorithm
Too-small memory grant
Hash or sort spill
Too many Key Lookups
Poor index choice
Unexpected Nested Loops
```

Where Cardinality Estimates Come From

SQL Server derives estimates mainly from:

EXAMPLE / FLOW

```
Statistics
Histograms
Density information
Constraints
Predicate information
Query transformations
```

Statistics histograms are one of the major inputs to cardinality estimation.

Index Seek

Definition

An **Index Seek** means SQL Server can navigate an index to find a particular value or range instead of reading the entire index.

Example index:

SQL

```
CREATE INDEX IX_Gift_ConstituentId
ON fundraising.Gift(ConstituentId);
```

Query:

```sql
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

Possible operator:

```
Index Seek
```

How It Works

Conceptually:

```
ConstituentId = 1001
    →
Index Root
    →
Correct Branch
    →
Correct Leaf Pages
    →
Matching Rows
```

The optimizer can choose a seek when a useful index and suitable predicate allow efficient navigation.

When Index Seek Is Usually Good

A seek is particularly useful when the query needs a relatively small subset of rows.

Example:

```
Gift table = 20,000,000 rows

ConstituentId 1001 = 12 rows
```

Reading only the relevant index area can be far cheaper than reading millions of rows.

Important Rule

Do not use:

```
Index Seek = Good
```

as an absolute rule.

Suppose:

```
Index Seek returns 2,000,000 rows
```

and then performs:

```
2,000,000 Key Lookups
```

That plan can be much more expensive than a scan.

Always examine the full execution plan.

Index Scan

Definition

An **Index Scan** means SQL Server reads many or all leaf pages of an index.

A scan can happen because:

EXAMPLE / FLOW

```
Most rows are required

No useful seekable index exists

Predicate cannot efficiently seek

Optimizer determines scanning is cheaper
```

Example

SQL

```
SELECT
    SUM(TotalAmount)
FROM fundraising.Gift;
```

Every gift amount must contribute to the total.

SQL Server may reasonably scan an index or table.

Flow

EXAMPLE / FLOW

```
Read Gift pages
    →
Read TotalAmount
    →
SUM values
    →
Return result
```

Index Scan Is Not Automatically Bad

Suppose:

EXAMPLE / FLOW

```
Gift table = 1,000,000 rows

Query needs = 900,000 rows
```

Performing 900,000 separate lookup operations would probably be inefficient.

A scan can be the correct plan.

SQL Server's own execution-plan documentation notes that when many or all rows are required, scanning can be the efficient access method.

When to Investigate a Scan

A scan deserves investigation when:

EXAMPLE / FLOW

```
Table is very large
+
Query returns very few rows
+
Many pages are being read
```

Example:

SQL

```
SELECT *
FROM fundraising.Gift
WHERE ConstituentId = 5001;
```

If only five rows match but SQL Server scans 20 million rows, check:

EXAMPLE / FLOW

```
Missing index
SARGability
Data type conversion
Statistics
Composite index column order
Parameter behavior
```

Index Seek vs Index Scan

Example:

EXAMPLE / FLOW

```
20,000,000 gifts
```

Query A:

SQL

```
WHERE GiftId = 10001
```

Maybe:

EXAMPLE / FLOW

```
Index Seek
```

because one row is required.

Query B:

SQL

```
SELECT SUM(TotalAmount)
FROM fundraising.Gift;
```

Maybe:

EXAMPLE / FLOW

```
Index Scan
```

because almost all data must be read.

The right question is not:

> ! Did SQL Server use a seek?

The better question is:

> ! Did SQL Server access the amount of data efficiently for this query?

Key Lookup

Definition

A **Key Lookup** happens when SQL Server finds rows through a nonclustered index but that index does not contain every column required by the query.

SQL Server then uses the clustered index key to retrieve the missing columns from the clustered index.

Example

Index:

```sql
CREATE INDEX IX_Gift_ConstituentId
ON fundraising.Gift(ConstituentId);
```

Query:

```sql
SELECT
    ConstituentId,
    GiftDate,
    TotalAmount,
    Notes
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

The index can find:

```
ConstituentId
```

but it does not necessarily contain all requested values:

```
GiftDate
TotalAmount
Notes
```

Possible plan:

```
Index Seek
    →
Key Lookup
    →
Nested Loops
    →
SELECT
```

Key Lookup Flow

```
Seek Nonclustered Index
    →
Get Clustered Key
    →
Find Base Row
    →
Retrieve Missing Columns
    →
Return Row
```

A Key Lookup normally appears with a Nested Loops operation because SQL Server may perform the lookup once for each qualifying outer row.

When Key Lookup Is Fine

Suppose:

```
Index Seek returns 4 rows
```

Four lookups are normally inexpensive.

There is usually no reason to redesign the entire indexing strategy merely because a Key Lookup appears.

When Key Lookup Becomes Expensive

Suppose:

EXAMPLE / FLOW

```
Index Seek returns 300,000 rows
```

and:

EXAMPLE / FLOW

```
Key Lookup executes 300,000 times
```

That can generate enormous I/O.

Possible improvement:

SQL

```sql
CREATE INDEX IX_Gift_ConstituentId
ON fundraising.Gift(ConstituentId)
INCLUDE
(
    GiftDate,
    TotalAmount
);
```

Now the important query may become covered by the index.

Sort Operator — Definition

A **Sort** operator orders incoming rows.

SQL Server may need a sort for:

EXAMPLE / FLOW

```
ORDER BY
GROUP BY
DISTINCT
Merge Join
Window functions
Some aggregate operations
```

Example:

SQL

```sql
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
ORDER BY GiftDate;
```

Possible flow:

EXAMPLE / FLOW

```
Read Gift
    →
Sort by GiftDate
    →
Return Rows
```

Sort Cost

Sorting can consume:

EXAMPLE / FLOW

```
CPU
Memory
Time
tempdb
```

especially for large row sets.

If SQL Server receives enough memory, the sort may remain in memory.

If the memory grant is insufficient, part of the sort may spill to `tempdb`.

Sort Spill

Conceptually:

EXAMPLE / FLOW

```
Rows
    →
Sort requires 500 MB
    →
Only 100 MB available
    →
Extra data written to tempdb
    →
Sort Spill
```

A spill can significantly increase execution time.

An actual execution plan can expose runtime warnings related to these operations.

Avoiding an Unnecessary Sort

Suppose:

SQL

```sql
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = @ConstituentId
ORDER BY GiftDate DESC;
```

An index such as:

SQL

```sql
CREATE INDEX IX_Gift_Constituent_GiftDate
ON fundraising.Gift
(
    ConstituentId,
    GiftDate DESC
)
INCLUDE
(
    TotalAmount
);
```

may allow SQL Server to retrieve rows already in the required order.

Possible plan:

EXAMPLE / FLOW

```
Index Seek
    →
Return Rows
```

instead of:

EXAMPLE / FLOW

```
Index Seek
    →
Sort
    →
Return Rows
```

Do not create an index merely to eliminate every Sort operator, though. The complete workload matters.

Join Algorithms

When SQL contains a logical join such as:

```sql
INNER JOIN
```

SQL Server still needs a **physical method** for performing that join.

The three important physical join algorithms are:

```
Nested Loops
Hash Match
Merge Join
```

The optimizer chooses between them according to estimated row counts, indexes, sorting, data size, and cost.

Nested Loops

Definition

Nested Loops takes rows from one input and searches the other input for matching rows.

Think:

```
For each outer row
    →
Find matching inner rows
```

Flow

```
Outer Row 1 → Search Inner → Matches

Outer Row 2 → Search Inner → Matches

Outer Row 3 → Search Inner → Matches
```

FundraisingDb Example

```sql
SELECT
    c.DisplayName,
    g.TotalAmount
FROM constituent.Constituent AS c
INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId = c.ConstituentId
WHERE c.ConstituentId = 1001;
```

If only one constituent is found and `Gift` has an index on `ConstituentId`, SQL Server might use:

```
Constituent Seek
    →
Nested Loops
    →
Gift Index Seek
```

When Nested Loops Works Well

Nested Loops is usually effective when:

EXAMPLE / FLOW

```
Outer input is small
+
Inner input has a useful index
```

Microsoft's join documentation describes Nested Loops as especially effective for small outer inputs with an indexed inner side.

Potential Problem

Suppose the optimizer estimates:

EXAMPLE / FLOW

```
Outer Rows = 10
```

but actually receives:

EXAMPLE / FLOW

```
Outer Rows = 1,000,000
```

The inner side may now execute one million times.

That can make Nested Loops extremely expensive.

This is why cardinality estimates matter.

Hash Match

Definition

A **Hash Match** uses a hash table to match rows.

For a hash join, SQL Server generally:

EXAMPLE / FLOW

```
Reads one input
    →
Builds hash table
    →
Reads second input
    →
Computes hash
    →
Finds matching rows
```

Simple Example

SQL

```
SELECT
    c.DisplayName,
    g.TotalAmount
FROM constituent.Constituent AS c
INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId = c.ConstituentId;
```

If both sides contain many rows and useful ordered/index access is not available, SQL Server might choose a Hash Match.

Hash Join Flow

EXAMPLE / FLOW

```
Smaller Input
    →
Build Hash Table
    →
Larger Input
```

```
    →
Probe Hash Table
    →
Return Matches
```

When Hash Match Is Useful

Hash joins are often useful for:

EXAMPLE / FLOW

```
Large data sets
Large unsorted inputs
Equality joins
Data warehouse/reporting queries
Cases without useful supporting indexes
```

Hash Match Is Not Only for Joins

`Hash Match` can also perform operations such as:

EXAMPLE / FLOW

```
Aggregation
DISTINCT-like processing
```

because hashing can efficiently group equal values.

Hash Spill

A hash operation requires memory.

If SQL Server expects:

EXAMPLE / FLOW

```
100,000 rows
```

but receives:

EXAMPLE / FLOW

```
10,000,000 rows
```

the requested memory may be insufficient.

Then parts of the hash operation may spill to `tempdb`.

Possible flow:

EXAMPLE / FLOW

```
Bad Estimate
    →
Small Memory Grant
    →
Large Actual Input
    →
Hash Spill
    →
tempdb I/O
    →
Slower Query
```

Merge Join

Definition

A **Merge Join** efficiently joins two inputs when both are ordered on their join columns.

Conceptually:

EXAMPLE / FLOW

```
Sorted Input A
+
Sorted Input B
    →
Walk Through Both Inputs
    →
Match Equal Keys
```

Example

Join:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,
    g.GiftId,
    g.TotalAmount
FROM constituent.Constituent AS c
INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId = c.ConstituentId;
```

If both inputs are already available in `ConstituentId` order, a Merge Join can be efficient.

Merge Join Flow

EXAMPLE / FLOW

```
ConstituentId-sorted Constituent
            +
ConstituentId-sorted Gift
            →
        Merge Join
            →
        Matching Rows
```

When Merge Join Works Well

It can work very well when:

EXAMPLE / FLOW

```
Both inputs are large
+
Both are already sorted on join keys
```

Microsoft documents Merge Join as particularly effective when sorted inputs are already available and an additional explicit sort is unnecessary.

Possible Problem

If the inputs are not sorted:

EXAMPLE / FLOW

```
Input A → Sort
Input B → Sort
        → Merge Join
```

The sorting cost may make another join algorithm cheaper.

Nested Loops vs Hash Match vs Merge Join

Nested Loops	Hash Match	Merge Join
Excellent for small outer input	Good for large unsorted sets	Good for large sorted sets

Nested Loops	Hash Match	Merge Join
Benefits strongly from inner index	Builds hash structure	Requires ordered inputs
Can be very fast for OLTP lookups	Often seen in large queries	Can efficiently stream matching rows
Can become costly with many outer rows	Uses memory	Sort may be required
Repeated inner operations	Can spill to tempdb	Works best with useful order/indexes

Do not force one join type merely because you believe one is always faster.

The optimizer chooses based on estimated cost.

Statistics

Definition

Statistics describe the distribution of data values.

The optimizer uses them primarily to estimate:

EXAMPLE / FLOW

```
How many rows will match?
```

This estimate then influences:

EXAMPLE / FLOW

```
Index choice
Join algorithm
Join order
Memory grant
Parallelism
Overall execution plan
```

SQL Server statistics contain information used directly for cardinality estimation.

Statistics Object

A statistics object can contain:

EXAMPLE / FLOW

```
Header
Histogram
Density information
```

Histogram

The histogram describes the distribution of values in the **first statistics key column**.

SQL Server histograms contain at most **200 steps**.

Example conceptually for:

EXAMPLE / FLOW

```
ConstituentId
```

EXAMPLE / FLOW

```
1001 → many gifts
1002 → few gifts
1003 → very many gifts
1004 → one gift
```

The optimizer can use the histogram to understand that different values may return different row counts.

Important Multi-Column Statistics Rule

Suppose statistics exist on:

EXAMPLE / FLOW

```
(TenantId, ConstituentId, GiftDate)
```

The histogram is created only on:

EXAMPLE / FLOW

```
TenantId
```

the first key column.

SQL Server also stores density information for column prefixes such as:

EXAMPLE / FLOW

```
(TenantId)

(TenantId, ConstituentId)

(TenantId, ConstituentId, GiftDate)
```

This is another reason column order matters.

Index Statistics

When you create a normal index:

SQL

```
CREATE INDEX IX_Gift_Constituent
ON fundraising.Gift(ConstituentId);
```

SQL Server also creates statistics for that index key.

You do not usually need to manually create another identical statistics object.

SQL Server automatically creates statistics for index keys.

Auto Create Statistics

SQL Server can automatically create statistics on predicate columns when needed.

The database option is:

EXAMPLE / FLOW

```
AUTO_CREATE_STATISTICS
```

For most databases, keeping automatic statistics creation enabled is recommended.

Auto Update Statistics

As data changes:

EXAMPLE / FLOW

```
INSERT
UPDATE
DELETE
```

statistics can become stale.

SQL Server normally has:

EXAMPLE / FLOW

```
AUTO_UPDATE_STATISTICS
```

enabled so statistics can be refreshed when needed.

Updating Statistics Manually

Example:

```sql
UPDATE STATISTICS fundraising.Gift;
```

Or specific statistics:

```sql
UPDATE STATISTICS fundraising.Gift IX_Gift_ConstituentId;
```

Do not blindly update all statistics constantly.

Updating statistics consumes resources and can trigger recompilation, so maintenance frequency should be based on workload requirements rather than a rule such as "update everything every hour."

Viewing Statistics

You can inspect statistics using:

```sql
DBCC SHOW_STATISTICS
(
    'fundraising.Gift',
    'IX_Gift_ConstituentId'
);
```

This can show:

```
STAT_HEADER
DENSITY_VECTOR
HISTOGRAM
```

SARGability Definition

SARGable means a predicate is written in a form that allows SQL Server to efficiently use an index search argument.

In simple terms:

```
Can SQL Server use the indexed value directly to find a range?
```

SARGable Example

Suppose an index exists:

```sql
CREATE INDEX IX_Gift_GiftDate
ON fundraising.Gift(GiftDate);
```

Good predicate:

```sql
SELECT *
FROM fundraising.Gift
WHERE GiftDate >= '20260101'
  AND GiftDate <  '20270101';
```

SQL Server can potentially perform:

```
Index Seek
    →
2026 date range
```

Non-SARGable Example

SQL

```
SELECT *
FROM fundraising.Gift
WHERE YEAR(GiftDate) = 2026;
```

Here a function is applied to the indexed column.

Conceptually SQL Server may need to do something more like:

EXAMPLE / FLOW

```
Read GiftDate
    →
Calculate YEAR(GiftDate)
    →
Compare with 2026
```

rather than directly finding the date range in the index.

Functions and expressions in predicates can also make cardinality estimation harder in some situations.

Better Date Predicate

Instead of:

SQL

```
WHERE YEAR(GiftDate) = 2026
```

prefer:

SQL

```
WHERE GiftDate >= '20260101'
  AND GiftDate <  '20270101'
```

This preserves the original column in the search predicate.

Another SARGability Problem

Suppose:

EXAMPLE / FLOW

```
ConstituentId BIGINT
```

Avoid unnecessary conversions such as:

SQL

```
WHERE CONVERT(VARCHAR(20), ConstituentId) = '1001';
```

Prefer:

SQL

```
WHERE ConstituentId = 1001;
```

LIKE and SARGability

This pattern can often use the beginning of an index:

SQL

```
WHERE DisplayName LIKE 'John%'
```

because the beginning is known.

This pattern is more difficult:

```sql
WHERE DisplayName LIKE '%John%'
```

because SQL Server does not know where inside the string `John` begins.

A normal B-tree index is not naturally organized for arbitrary middle-of-string searches.

Parameter Sniffing

Definition

When a parameterized query or stored procedure is compiled, SQL Server can use the parameter value available at compilation time to estimate row counts.

This behavior is commonly called **parameter sniffing**.

Parameter sniffing itself is useful because the optimizer can build a plan appropriate for the observed parameter value.

The problem occurs when different parameter values need very different execution plans.

Example Procedure

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_GetGiftsByConstituent
    @ConstituentId BIGINT
AS
BEGIN
    SELECT
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
```

Imagine:

```
Constituent 1001 → 2 gifts

Constituent 2001 → 2,000,000 gifts
```

These parameter values represent very different workloads.

Parameter Sniffing Flow

First call:

```sql
EXEC fundraising.usp_GetGiftsByConstituent
    @ConstituentId = 1001;
```

Possible flow:

```
Parameter = 1001
    →
Optimizer sees tiny result
    →
Creates plan for small result
    →
Plan cached
```

Later:

```sql
EXEC fundraising.usp_GetGiftsByConstituent
    @ConstituentId = 2001;
```

Possible flow:

```
Parameter = 2001
    →
Cached plan reused
    →
Plan designed for tiny result
    →
But 2,000,000 rows returned
    →
Poor performance
```

Why Plan Reuse Exists

Plan compilation itself costs resources.

SQL Server therefore caches execution plans so repeated statements can often reuse an existing plan.

```
Compile Once
    →
Cache Plan
    →
Reuse Plan
```

This is normally beneficial.

The difficulty appears when:

```
One Plan
```

cannot efficiently serve very different data distributions.

Parameter Sensitivity

A more accurate description of the performance problem is often:

```
Parameter-Sensitive Plan Problem
```

rather than assuming parameter sniffing itself is always bad.

The underlying issue is commonly:

```
Skewed Data Distribution
+
One Cached Plan
+
Very Different Parameter Selectivity
```

Parameter Sensitive Plan Optimization

Modern SQL Server includes **Parameter Sensitive Plan (PSP) optimization**.

SQL Server 2022 and later can maintain multiple active cached plan variants for eligible parameter-sensitive queries when the feature requirements are satisfied.

This allows different parameter ranges to use different plans.

Conceptually:

EXAMPLE / FLOW

```
Parameterized Query
    →
Dispatcher Plan
    →
Small parameter range → Plan A
Medium range          → Plan B
Large range           → Plan C
```

Since your learning baseline is SQL Server 2025, this modern behavior is important to understand.

Do Not Immediately Disable Parameter Sniffing

When parameter-sensitive behavior is suspected, first investigate:

EXAMPLE / FLOW

```
Statistics
Data skew
Actual vs estimated rows
Execution plans
Query Store history
Indexes
PSP optimization
```

Do not immediately add hints everywhere.

Microsoft recommends optimizer hints and Query Store hints only when necessary after normal tuning possibilities are considered.

Query Store

Definition

Query Store keeps historical information about queries, plans, and runtime performance.

It helps answer questions such as:

EXAMPLE / FLOW

```
Which queries consume the most CPU?

Which queries became slower?

Did the execution plan change?

Which plan was used yesterday?

How often does this query execute?

Which plan has better runtime behavior?
```

Query Store stores query text, plans, and aggregated runtime statistics.

Query Store Flow

EXAMPLE / FLOW

```
Query Compiles
    →
Query Text + Plan captured
    →
Query Executes
    →
Runtime Statistics captured
```

```
    →
Stored historically
    →
DBA analyzes later
```

Why Query Store Is Important

Without Query Store, imagine:

EXAMPLE / FLOW

```
Monday
Query = 100 ms
```

Then:

EXAMPLE / FLOW

```
Tuesday
Query = 8 seconds
```

You look Tuesday and see only the current plan.

The old good plan may no longer be easily visible.

With Query Store:

EXAMPLE / FLOW

```
Monday Plan A → 100 ms

Tuesday Plan B → 8 sec
```

You can investigate the plan change.

Query Store and Plan Regression

A **plan regression** means a query receives a new execution plan that performs worse than an earlier one.

Conceptually:

EXAMPLE / FLOW

```
Query
    →
Plan A
    →
Fast
```

later:

EXAMPLE / FLOW

```
Query
    →
Plan B
    →
Slow
```

Query Store can help detect this type of historical performance change.

Query Store and Plan Forcing

Query Store supports plan forcing.

Example conceptually:

EXAMPLE / FLOW

```
Query 55
    →
Plan 10 = good
Plan 20 = bad
    →
Force Plan 10
```

T-SQL can use procedures such as:

SQL

```sql
EXEC sys.sp_query_store_force_plan
    @query_id = 55,
    @plan_id = 10;
```

But plan forcing should not be the first tuning technique.

First investigate why the regression happened.

Possible causes include:

EXAMPLE / FLOW

```
Statistics changes
Index changes
Data growth
Parameter sensitivity
Different compatibility level
Schema changes
```

SET STATISTICS IO

Definition

`SET STATISTICS IO` tells SQL Server to report I/O information for executed queries.

Syntax:

SQL

```sql
SET STATISTICS IO ON;
```

Run query:

SQL

```sql
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

Then disable:

SQL

```sql
SET STATISTICS IO OFF;
```

It reports values such as logical reads, physical reads, read-ahead reads, and scan count.

Logical Reads

A **logical read** means SQL Server accessed an 8-KB data page from the buffer cache.

Example:

EXAMPLE / FLOW

```
logical reads 20
```

means SQL Server accessed 20 pages from cache for that object.

Logical reads are one of the most useful measurements when comparing query designs.

Physical Reads

A **physical read** means SQL Server had to retrieve a page from storage because it was not already available in memory.

Example:

EXAMPLE / FLOW

```
physical reads 4
```

Do not rely only on physical reads for query comparisons.

Repeated testing may show:

EXAMPLE / FLOW

```
physical reads = 0
```

simply because the required pages are already cached.

Logical reads are often more stable for comparing how much data SQL Server had to access.

Read-Ahead Reads

SQL Server may predict that pages will soon be required and load them before the execution operator requests them.

These are reported as:

EXAMPLE / FLOW

```
read-ahead reads
```

Read-ahead is useful particularly for larger sequential scans.

Scan Count

`Scan count` shows the number of scan/seek operations initiated at the leaf level while obtaining the result.

Do not simply interpret:

EXAMPLE / FLOW

```
Scan count 1 = bad
```

or:

EXAMPLE / FLOW

```
Scan count 0 = good
```

Its meaning depends on the access method and plan. Microsoft documents the exact behavior for unique seeks, scans, and repeated access.

STATISTICS IO Example

SQL

```
SET STATISTICS IO ON;

SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;

SET STATISTICS IO OFF;
```

Possible output:

EXAMPLE / FLOW

```
Table 'Gift'.
Scan count 1,
logical reads 4,
physical reads 0,
read-ahead reads 0.
```

Interpretation:

EXAMPLE / FLOW

```
Gift table/index accessed
4 pages read from cache
No storage reads required
```

Comparing Two Queries with STATISTICS IO

Query A:

EXAMPLE / FLOW

```
logical reads = 48,000
```

Query B:

EXAMPLE / FLOW

```
logical reads = 120
```

If both return the same required result, Query B is doing dramatically less page work.

This is often a stronger performance signal than simply saying:

EXAMPLE / FLOW

```
Query A took 800 ms
Query B took 500 ms
```

because elapsed time can vary due to server load and waits.

SET STATISTICS TIME

Definition

`SET STATISTICS TIME` displays timing information for SQL statement parsing, compilation, and execution.

Syntax:

SQL

```
SET STATISTICS TIME ON;
```

Microsoft documents it as reporting milliseconds used to parse, compile, and execute statements.

STATISTICS TIME Example

SQL

```
SET STATISTICS TIME ON;

SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;

SET STATISTICS TIME OFF;
```

Possible output:

EXAMPLE / FLOW

```
SQL Server Execution Times:
CPU time = 20 ms,
elapsed time = 35 ms.
```

CPU Time

EXAMPLE / FLOW

```
CPU time = 20 ms
```

means SQL Server processors spent approximately that amount of CPU processing on the query.

High CPU may come from operations such as:

EXAMPLE / FLOW

```
Large scans
Sorting
Hashing
Complex calculations
Poor joins
Large aggregations
```

Elapsed Time

EXAMPLE / FLOW

```
elapsed time = 35 ms
```

means total wall-clock execution duration.

Elapsed time can include:

EXAMPLE / FLOW

```
CPU work
I/O waits
Blocking
Memory waits
Parallel processing effects
Other waits
```

CPU Time vs Elapsed Time

Example:

EXAMPLE / FLOW

```
CPU     = 100 ms
Elapsed = 5,000 ms
```

The query spent most of its total time waiting rather than actively consuming CPU.

Possible causes:

EXAMPLE / FLOW

```
Blocking
Disk I/O
Network
Resource waits
Memory
```

Another example:

EXAMPLE / FLOW

```
CPU     = 3,000 ms
Elapsed = 1,000 ms
```

This can happen with a parallel query because multiple workers consume CPU concurrently.

Using STATISTICS IO and TIME Together

For performance testing:

SQL

```sql
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;

SET STATISTICS TIME OFF;
SET STATISTICS IO OFF;
```

Microsoft explicitly recommends using these together when investigating server-side query resource use.

Practical Query-Tuning Example

Suppose this query is slow:

SQL

```sql
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

And suppose no useful index exists.

Possible plan:

EXAMPLE / FLOW

```
Clustered Index Scan
    →
Filter ConstituentId = 1001
    →
Return 20 rows
```

Statistics:

EXAMPLE / FLOW

```
Gift rows      = 10,000,000
Rows returned  = 20
Logical reads  = 80,000
```

This deserves investigation.

Create Appropriate Index

SQL

```sql
CREATE INDEX IX_Gift_ConstituentId
ON fundraising.Gift(ConstituentId)
INCLUDE
(
```

```
    GiftDate,
    TotalAmount
);
```

Now possible plan:

EXAMPLE / FLOW

```
Index Seek
    →
Return 20 rows
```

Possible measurements:

EXAMPLE / FLOW

```
Logical reads = 4
```

The improvement did not happen because:

EXAMPLE / FLOW

```
Seek is magically better
```

It happened because SQL Server could now read only the small amount of data required for the query.

Practical Cardinality Problem

Suppose:

SQL

```
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE CampaignId = 100;
```

Execution plan:

EXAMPLE / FLOW

```
Estimated Rows = 100
Actual Rows    = 800,000
```

Investigation should consider:

EXAMPLE / FLOW

```
Statistics stale?

Strongly skewed CampaignId values?

Parameter-sensitive query?

Statistics missing?

Predicate transformation?

Implicit conversion?

Different parameter value used during compilation?
```

Do not immediately create another index before understanding the estimation problem.

Practical Nested Loops Problem

Plan:

EXAMPLE / FLOW

```
Campaign Seek
    →
Nested Loops
    →
Gift Index Seek
```

Estimated:

EXAMPLE / FLOW

```
Campaign rows = 1
Gift matches  = 10
```

Actual:

EXAMPLE / FLOW

```
Gift matches = 1,500,000
```

Nested Loops may now execute its inner operations far more often than expected.

The root problem might not be Nested Loops itself.

The real problem may be:

EXAMPLE / FLOW

```
Incorrect cardinality estimate
```

which caused the optimizer to choose Nested Loops.

Practical Hash Match Example

Query:

SQL

```
SELECT
    g.CampaignId,
    SUM(g.TotalAmount)
FROM fundraising.Gift AS g
GROUP BY g.CampaignId;
```

Possible plan:

EXAMPLE / FLOW

```
Gift Scan
    →
Hash Match Aggregate
    →
Return Campaign Totals
```

For a very large unordered input, a Hash Match Aggregate can be completely reasonable.

Do not assume:

EXAMPLE / FLOW

```
Hash Match = bad
```

Practical Sort Example

Query:

SQL

```
SELECT
    GiftId,
    GiftDate,
    TotalAmount
```

```
FROM fundraising.Gift
WHERE ConstituentId = 1001
ORDER BY GiftDate DESC;
```

Possible plan:

EXAMPLE / FLOW

```
Index Seek on ConstituentId
    →
Sort GiftDate DESC
    →
Return rows
```

Possible alternative index:

SQL

```
CREATE INDEX IX_Gift_Constituent_GiftDate
ON fundraising.Gift
(
    ConstituentId,
    GiftDate DESC
)
INCLUDE
(
    TotalAmount
);
```

Possible new plan:

EXAMPLE / FLOW

```
Index Seek
    →
Return Already Ordered Rows
```

Whether this new index is worth maintaining depends on how important and frequent the query is.

How to Read a Performance Problem

3D DIAGRAM



Do not begin with:

EXAMPLE / FLOW

```
I see a Scan.
Create an index.
```

Use a more complete process:

EXAMPLE / FLOW

```
Slow Query
    →
Check Actual Execution Plan
    →
Check Actual vs Estimated Rows
    →
Check STATISTICS IO
    →
Check STATISTICS TIME
    →
Check Index Access
    →
Check Join Operators
    →
Check Key Lookup repetitions
    →
Check Sort/Hash warnings
    →
Check Statistics
    →
Check SARGability
    →
Check Parameter Sensitivity
    →
Check Query Store History
    →
Make targeted change
    →
Measure again
```

Important Execution Plan Properties to Check

When you click an operator in SSMS, useful properties include:

EXAMPLE / FLOW

```
Actual Number of Rows
Estimated Number of Rows
Actual Number of Executions
Estimated Number of Rows Per Execution
Predicate
Seek Predicates
Output List
Object
Ordered
Estimated I/O Cost
Estimated CPU Cost
Warnings
Memory Grant information
```

Do not judge an operator only from its graphical percentage.

The plan cost percentages are based on optimizer estimates, not a direct stopwatch measurement of every operator's actual runtime.

Important Performance Relationships

Statistics → Cardinality

EXAMPLE / FLOW

```
Statistics
    →
Estimate Row Count
```

Cardinality → Plan Choice

EXAMPLE / FLOW

```
Estimated Rows
    →
Seek / Scan
Join Type
Memory
Sort
Parallelism
```

Index → Data Access

EXAMPLE / FLOW

```
Useful Index
    →
Efficient Seek / Ordered Scan
```

Poor SARGability → Harder Index Access

EXAMPLE / FLOW

```
Function on Indexed Column
    →
Cannot efficiently navigate index
    →
More data may be read
```

Parameter Sensitivity → Plan Reuse Problem

EXAMPLE / FLOW

```
Parameter Value A
    →
Plan cached
    →
Parameter Value B
    →
Same plan may not fit
```

Query Store → Historical Evidence

EXAMPLE / FLOW

```
Old Plan + Runtime
        vs
New Plan + Runtime
```

Common Beginner Mistakes

Looking only for Index Scans

A scan can be correct.

Ask how much data the query actually needs.

Assuming every Index Seek is good

A seek returning millions of rows plus millions of lookups can be expensive.

Automatically eliminating every Key Lookup

A lookup executed three times is usually not a major problem.

A lookup executed 500,000 times may be.

Ignoring Actual vs Estimated Rows

Large estimation differences can explain many bad plan decisions.

Creating indexes before checking statistics

Sometimes the required index already exists but SQL Server has poor estimates because statistics are stale or insufficient.

Updating every statistic constantly

Statistics maintenance has a cost and can trigger recompilation.

Use it when justified.

Using functions unnecessarily on indexed columns

Example:

```sql
WHERE YEAR(GiftDate) = 2026
```

may prevent efficient range seeking.

Prefer:

```sql
WHERE GiftDate >= '20260101'
  AND GiftDate < '20270101'
```

Treating parameter sniffing as always bad

Plan reuse and parameter sniffing are normally beneficial.

Investigate only when different parameter values require significantly different execution strategies.

Immediately using query hints

Hints can force optimizer behavior that later becomes incorrect when data changes.

Use them only after understanding the actual problem.

Practical Beginner-to-Advanced Analysis Example

Start with:

```sql
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT
    c.DisplayName,
    g.GiftDate,
    g.TotalAmount
FROM constituent.Constituent AS c
INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId = c.ConstituentId
WHERE c.TenantId = 1
  AND g.GiftDate >= '20260101'
  AND g.GiftDate <  '20270101';

SET STATISTICS TIME OFF;
SET STATISTICS IO OFF;
```

Then examine the **Actual Execution Plan**.

Ask:

EXAMPLE / FLOW

```
How many rows were estimated?
How many actually returned?

Was Gift scanned or sought?

Was Constituent scanned or sought?

Which join was selected?

Was Nested Loops executed too many times?

Was there a Hash Match?

Was there an unnecessary Sort?

Were Key Lookups repeated heavily?

Were there spill warnings?

How many logical reads occurred?

How much CPU was consumed?

What was elapsed time?

Are statistics accurate?

Are predicates SARGable?

Could parameter values produce very different row counts?

Does Query Store show a previous better plan?
```

That is the correct way to move from beginner-level execution-plan reading into real SQL Server performance tuning.

Complete Performance Flow

EXAMPLE / FLOW

```
Application sends SQL
    →
SQL Server parses query
    →
Query Optimizer reads
Query + Schema + Indexes + Statistics + Parameters
    →
Cardinality Estimator predicts rows
    →
Optimizer compares candidate strategies
    →
Execution Plan selected
    →
Operators execute

Seek / Scan
    →
Nested Loops / Hash / Merge
    →
Sort / Aggregate
    →
Lookup if required
    →
Result returned
    →
Runtime information available
    →
Actual Plan + STATISTICS IO + STATISTICS TIME
    →
Query Store keeps historical performance
```

This is the foundation for understanding SQL Server performance from beginner level through advanced query tuning.